27-Day Fullstack Project Plan: Multi-Tenant Workflow Automation System

Day 1 (Mar 4, 2026): Setup Development Environment

- **Objective:** Establish a working development environment and project skeleton. Ensure Java, Node.js, and tools are installed so coding can begin.
- **File Locations & Setup:** Install JDK 17 and Node.js 18 (required versions for Spring Boot and React). Create a Git repository and use Spring Initializr (https://start.spring.io) to generate a new Spring Boot project with dependencies like **Spring Web**, **Spring Data JPA**, and **PostgreSQL Driver**. Verify the project folder (e.g. src/main/java, src/main/resources, pom.xml or build.gradle) is properly created.
- **Core Concepts:** Familiarize yourself with the fullstack toolchain: Spring Boot for the backend and React for the frontend. Understand basic CRUD app structure (as in the Okta guide). Ensure IDE (IntelliJ/Eclipse) and code editor (VSCode) are configured.
- **Implementation:** Follow a quick start tutorial to initialize the Spring Boot backend (e.g. generate the project and import into your IDE). Run mvn spring-boot:run (or Gradle equivalent) to verify the app starts. In parallel, create a new React app (npx create-react-app) in a separate folder and confirm npm start runs. Set up a **.gitignore** and make the first commit with this boilerplate.
- **Consequences of Skipping:** Without a properly configured environment, no code can run. Skipping initial setup risks hidden configuration issues later (broken builds, version mismatches) and wastes time fixing basic tooling.

Day 2 (Mar 5, 2026): Multi-Tenancy Architecture & Database Design

- **Objective:** Research and decide on a multi-tenant isolation strategy. Plan how the database will separate tenant data (e.g. shared schema with RLS). Establish a high-level system architecture diagram.
- **File Locations & Setup:** Create a design document (in /docs/ or README) outlining the chosen multi-tenancy model. If using a shared database, prepare a new database schema (e.g. CREATE SCHEMA multitenant; in PostgreSQL). Edit Spring's configuration (application.yml or application.properties) to include the connection to the new database/schema.
- **Core Concepts:** Learn what multi-tenancy means: a *multi-tenant application* is one where separate customers ("tenants") share the same application instance yet have isolated data. Common models are: **(1)** separate databases per tenant, **(2)** shared DB but separate schemas per tenant, or **(3)** shared schema with a tenant-ID column. The shared-schema approach is simplest to start with, but requires strict data isolation (e.g. via PostgreSQL RLS).
- **Implementation:** Document your chosen strategy (likely shared DB with a tenant_id/tenant_name column on each table) in the design doc. Write initial SQL scripts to create tenant identifiers and test tables. For example, prepare

multitenant.create_schema.sql to create the multitenant schema and a tenants table. Ensure each table will include a tenant field. If using RLS, include ALTER TABLE ... ENABLE ROW LEVEL SECURITY;. Reference best practices like ensuring every query filters by tenant (conceptually done by RLS).

- **Consequences of Skipping:** If you don't plan multi-tenancy from the start, you risk later complications and data leaks. Poorly designed tenancy means one tenant's data could be exposed to another, which is a serious security breach. Without a clear strategy, development will stall when it's time to implement tenant separation.

Day 3 (Mar 6, 2026): Spring Boot Backend Initialization & RLS Setup

- **Objective:** Set up the Spring Boot backend project and connect it to PostgreSQL with Row-Level Security (RLS) enabled. Begin implementing tenant context passing.
- **File Locations & Setup:** In src/main/resources, configure application.yml (or .properties) with the PostgreSQL datasource (URL, user, password). Create an SQL script file (e.g. db-init.sql in src/main/resources) that creates the tenants table and sample users, enabling RLS policies (using ALTER TABLE ... ENABLE ROW LEVEL SECURITY).
- **Core Concepts:** Understand Row-Level Security (RLS) in PostgreSQL. RLS lets the database automatically filter rows by tenant without needing WHERE clauses in every query. You define security policies that act like an automatic WHERE filter on each table. For example, CREATE POLICY tenant_isolation ON customers USING (tenant_name = current_user); ensures each query only returns rows matching the current database user (when using separate DB roles per tenant). RLS can also use a session variable (via current_setting) for tenant context.
- **Implementation:** Initialize the Postgres database (locally or Docker) and run the schema creation script. For a start, create at least two dummy tenants (e.g. roles tenant_a, tenant_b with passwords). In Spring Boot, add code to set the tenant context per request (e.g., an HTTP filter that reads an X-Tenant-ID header and stores it in a thread-local). You may also implement a CurrentTenantIdentifierResolver (if using Hibernate multi-tenancy). Test that connecting as tenant_a returns only tenant_a's rows by default.
- **Consequences of Skipping:** Without initializing RLS and the database schema now, you will have no effective tenant isolation. Skipping this setup means any data access code will likely expose other tenants' data, undermining the core multi-tenant requirement.

Day 4 (Mar 7, 2026): Define Data Model – Entities & JPA Repositories

- **Objective:** Design and implement the core domain model entities and JPA repositories for tenants, users, workflows, and tasks.

- **File Locations & Setup:** Under src/main/java/.../model, create Entity classes: e.g. Tenant, User, WorkflowDefinition, WorkflowInstance, Task, etc. Under src/main/java/.../repository, create Spring Data JPA interfaces for each (e.g. TenantRepository). Ensure your application.yml points to the correct packagesToScan so Spring detects these entities.
- **Core Concepts:** Each entity should include a tenant identifier field that matches your RLS policy column (e.g. tenant_name). In a shared-schema model, every table needs the tenant column so RLS can enforce isolation. Understand basic JPA annotations (@Entity, @Id, @ManyToOne, etc.) for modeling relationships: for example, a WorkflowInstance may reference a User (assignedTo) and contains many Task entities.
- **Implementation:** Create the entity classes with fields, constructors, and getters/setters. Use JPA annotations to map relationships (e.g. a WorkflowDefinition might have a list of TaskDefinition templates). Also create repository interfaces extending JpaRepository. Write a sample unit test or a CommandLineRunner to persist a test tenant and user to verify the mappings.
- **Consequences of Skipping:** Without a properly defined data model and repositories, there's no foundation for the application's logic. Skipping entity/repository creation means you cannot persist or query any business data.

Day 5 (Mar 8, 2026): Basic RESTful API Endpoints (Backend)

- **Objective:** Develop core REST controllers for CRUD operations on your entities (e.g. create/update workflows, tasks, tenants).
- **File Locations & Setup:** In src/main/java/.../controller, create classes like TenantController, WorkflowController, TaskController, each annotated with @RestController. Define routes (e.g. /api/tenants, /api/workflows) and use your repositories to implement methods.
- **Core Concepts:** Understand Spring MVC controllers and how to map HTTP methods (GET, POST, PUT, DELETE) to Java methods. Also, Spring Boot will automatically apply your RLS policies on repository queries. Use @RequestBody for JSON payloads and @PathVariable for IDs. Ensure you apply tenant scoping by requiring the X-Tenant-ID header or token on each request (the filter from Day 3 should enforce this).
- **Implementation:** Write endpoints such as GET /api/tenants/{id}, POST /api/workflows, etc. For example, a POST to /api/workflows should read the JSON body into a WorkflowDefinition object, set its tenant_name from the current context, and save it. Test using Postman or CURL. Verify that when you attempt to access data without the correct tenant header, the filter rejects it (e.g. return 400 Bad Request for missing X-Tenant-ID).

- **Consequences of Skipping:** If no API endpoints exist, the frontend cannot use the backend. Skipping this leaves the application non-functional. Also, failing to tie endpoints to the tenant context (e.g. forgetting to pass the tenant ID to the service) risks leaking or misassigning data.

Day 6 (Mar 9, 2026): Exam Period – Documentation & Light Research

- **Objective:** Due to exams, focus on lighter, preparatory tasks: write documentation, review concepts, and plan upcoming work.
- **File Locations & Setup:** Update the project README with project overview, architecture decisions (including multi-tenancy strategy) and how to run the app. Create or expand design docs in a /docs folder, including an ER diagram.
- **Core Concepts:** Use this time to read more about PostgreSQL RLS, multi-tenant pitfalls, and React best practices. Skim articles on secure Spring Boot practices to prepare for later. If needed, study React Context API and theming ideas.
- **Implementation:** Organize what's been done: commit all code, annotate methods with comments, ensure consistency. Jot down user stories or refine requirements. Sketch basic UI wireframes on paper or with a tool (e.g. login screen, dashboard layouts).
- **Consequences of Skipping:** Even during exams, updating docs and planning prevents confusion later. Skipping these tasks can lead to disorganization or forgetting important details, making later implementation slower and error-prone.

Day 7 (Mar 10, 2026): Exam Period – Frontend Planning

- **Objective:** Continue with low-impact tasks: set up the frontend project structure and finalize tenant-context strategy on the UI side.
- **File Locations & Setup:** Initialize the React app if not done (e.g. src/, public/), and install any needed libraries (e.g. npm install axios react-router-dom). Ensure consistency with backend API paths.
- **Core Concepts:** Decide how to identify tenants in the UI. Commonly, the tenant ID might be part of login credentials or stored in local storage. Research implementing a global tenant context (e.g. using React Context or a global state library). Plan how to pass this context (for example, sending X-Tenant-ID on every API call).
- **Implementation:** Create a React Context (e.g. TenantContext) that will store the current tenant ID and any theming info. Write a stub login page or selection page where the user enters or selects a tenant. Verify that React can fetch tenant-specific data by passing the header on axios requests (you can hardcode a value and test a simple fetch).

- **Consequences of Skipping:** Without planning tenant context on the frontend, you risk building UI that cannot distinguish tenants. Skipping this makes implementing multi-tenancy in the UI much harder, causing rework later.

Day 8 (Mar 11, 2026): Exam Period – Workflow Concept and Data Prep

- **Objective:** Define the workflow domain in detail. Determine what a "workflow" and its "tasks" mean for this application. Prepare any necessary seed data.
- **File Locations & Setup:** Create or update WorkflowDefinition and TaskDefinition entities/data structures. Optionally prepare SQL seed scripts (create_data.sql) with sample workflows for testing.
- **Core Concepts:** Understand what automated workflows require (steps, triggers, conditions). Decide if tasks are manual steps or automated jobs (e.g. email, database update). Consider using a scheduling system or event-driven approach for executing workflows.
- **Implementation:** Draft sample workflow definitions, for instance a JSON or database entry that represents a sequence of steps. If time, enter sample data into the DB (e.g. one workflow with 3 tasks) so you have something to display later. Also finalize the naming convention for tables and columns (e.g. always include tenant_name column).
- **Consequences of Skipping:** If the workflow logic isn't clearly defined, development will stall later. Incomplete data models at this stage could require redesign. Ensuring sample data exists helps test UI and APIs.

Day 9 (Mar 12, 2026): Exam Period – Refine Design & Code Review

- **Objective:** Review all work so far. Refine any rough edges, and prepare to move into heavy coding again.
- **File Locations & Setup:** Walk through code in all packages (model, repository, service, controller, and React src). Fix any immediate syntax errors or warnings.
- **Core Concepts:** Ensure consistency in coding standards. Confirm that each entity has a tenant field. Review security filter logic and CORS settings (to allow React access).
- **Implementation:** Clean up code comments, add Javadoc if needed, and ensure each service method has error handling. If possible, update application.yml with profiles (e.g. dev, prod). Perform a quick smoke test of the backend APIs with Postman.
- **Consequences of Skipping:** Skipping review increases the chance of embedded bugs or inconsistencies. It's better to catch mistakes now than after more features have been built.

Day 10 (Mar 13, 2026): Implement Workflow Engine & Scheduling

- **Objective:** Build the core workflow processing logic, including any scheduling or state transitions. Ensure workflows can be started and progress through their tasks automatically.
- **File Locations & Setup:** In src/main/java/.../service create classes like WorkflowService and possibly a scheduled task runner (using Spring's @Scheduled or Quartz Scheduler). Ensure all new services are wired into Spring.
- **Core Concepts:** A workflow typically has a definition (sequence of steps) and instances (in-progress executions). Tasks might execute sequentially or in response to triggers. Using a scheduling library (e.g. Quartz Scheduler) can help automate task execution at intervals. For example, you might schedule a job that periodically checks for "ready" workflows and advances them.
- **Implementation:** Implement methods such as startWorkflow(tenantId, workflowDefinitionId) that creates a WorkflowInstance. Add a scheduled method (e.g. annotated with @Scheduled(fixedDelay = 60000)) to scan for any pending workflow instances and execute the next step (or mark them complete). Use logs to print progress. If using Quartz, configure a job and trigger for workflows (this may be more advanced). Ensure RLS still applies when these methods use repositories.

- **Implementation (cont.):** Make sure to **set the tenant context** before any workflow logic runs. For example, if using current_setting('app.current_tenant') in RLS, execute a native query SET app.current_tenant = 'tenantA' at the start of each scheduled task so the database knows which tenant it's acting under. Test by creating a workflow under tenant A and ensure only tenant A's tasks run (tenant B's tasks should not appear).
- **Consequences of Skipping:** Without a workflow engine, your system won't automate anything – the core feature would be missing. Skipping the scheduling step means tasks will never advance on their own. Moreover, failure to integrate tenant context here could violate RLS and mix up data.

Day 11 (Mar 14, 2026): Business Logic – Task Execution & State Management

- **Objective:** Flesh out the business logic for workflow tasks: execution rules, state transitions, and outcomes. Implement the actions that occur when tasks are triggered.
- **File Locations & Setup:** Continue work in WorkflowService (or create TaskService) and controllers. Add or update service methods to handle task completion, skipping, or error states.
- **Core Concepts:** Each workflow task may have a type (e.g. data processing, sending notifications). Decide how to model task status (e.g. PENDING, IN_PROGRESS, COMPLETE). Ensure that after each task completes, the system knows which task is next. You may design this using a state machine or simply sequence logic.

- **Implementation:** Write code so that when the scheduled job runs, it fetches the next pending task (WHERE status='PENDING') and processes it (this could be a mock action, like logging "executing task X"). Update the task's status to COMPLETE and create the next pending task. If there's branching logic or conditions, encode those rules (e.g. if Task A has two possible next steps). Confirm each method uses tenant_name = current context to respect multi-tenancy.
- **Consequences of Skipping:** Skipping this means workflows will not actually do anything meaningful. The intern would have an unfinished product. Additionally, if you don't carefully propagate the tenant context in logic, tasks might operate on the wrong tenant's data.

Day 12 (Mar 15, 2026): (Optional) Implement Authentication/Authorization

- **Objective:** Secure the backend by adding an authentication layer so that only authorized users can access APIs. Tie users to tenants.
- **File Locations & Setup:** Add Spring Security to the backend (if not already). Create a SecurityConfig class in src/main/java/.../security. Set up a simple user store (in-memory or in database) with usernames, passwords, and roles. Associate each user with a tenant.
- **Core Concepts:** Authentication ensures only valid users access the system. For multi-tenancy, users should only belong to one tenant and see only that tenant's data. Common patterns include JWT tokens or basic auth for an internship project. Plan that after login, the app knows which tenant the user is (so it can send the tenant ID header on requests).
- **Implementation:** Configure HTTP security rules (e.g. require authentication for /api/**). Implement a login endpoint (/api/login) that issues a token or session. If using JWT, issue a token containing the tenant_id claim and have a filter to extract it. At minimum, secure endpoints with Basic Auth for testing (username/password corresponds to tenant).
- **Consequences of Skipping:** Without authentication, any API is open, which is not production-ready. However, if time is tight this can be noted as a limitation; but be aware it weakens the demo, as you'll have to bypass auth when calling APIs.

Day 13 (Mar 16, 2026): Finalize RLS and Multi-Tenant Security

- **Objective:** Review and solidify the database security policies. Ensure that RLS is correctly restricting data for all tables and all actions (SELECT, INSERT, UPDATE, DELETE).
- **File Locations & Setup:** Update your SQL initialization scripts (db-init.sql or similar) to include all ALTER TABLE ... ENABLE ROW LEVEL SECURITY and CREATE POLICY statements for each table. For example, use a session setting policy:

```
CREATE POLICY workflow_isolation ON multitenant.workflow_instance
    USING (tenant_name = current_setting('app.current_tenant')::VARCHAR);
```

as suggested in documentation.

- **Core Concepts:** Revisit how RLS policies work for writes vs. reads. If using session variables (current_setting), every new database connection (e.g. each service call) must set app.current_tenant. Be aware that pooled connections could leak context, so one approach is to use a separate login per tenant or explicitly reset the setting in each transaction. **Testing as Tenant vs. System:** Note that a superuser or the schema owner bypasses RLS unless FORCE ROW LEVEL SECURITY is used.
- **Implementation:** Execute or run the updated SQL scripts to (re)create policies. Test by logging into Postgres as a non-privileged role: attempt to query or modify data without setting app.current_tenant (you should see no results or errors). Then set app.current_tenant to a specific tenant and retry – now you should see that tenant's rows. Verify that inserts without the correct tenant_name are blocked by the policy.
- **Consequences of Skipping:** If RLS is not fully in place, tenant data can mix. For example, if an INSERT lacks the policy check, one tenant could insert data into another tenant's scope. Skipping final RLS checks undermines the application's security guarantees.

Day 14 (Mar 17, 2026): Backend Testing & Validation

- **Objective:** Write and run tests for the backend to verify functionality and multi-tenancy enforcement.
- **File Locations & Setup:** In src/test/java, create unit and integration tests. You might use Spring Boot Test with an in-memory or test database. Write tests for services and controllers (e.g. using @WebMvcTest for controllers). Include tests specifically for RLS: one test per tenant confirming isolation.
- **Core Concepts:** Automated tests ensure the logic works as expected and remains correct through changes. Test scenarios include: a tenant can create a workflow and retrieve it, and cannot see other tenants' workflows (simulate by setting app.current_tenant or mocking the filter). Also test edge cases (e.g. invalid tenant ID, missing headers).
- **Implementation:** For each controller endpoint, write a test that calls the endpoint with a mock request including the tenant header. Assert the correct response. For example, create a workflow under "tenant1" and query for workflows – you should see it. Then query for "tenant2" with the same call and assert it returns empty or forbidden. Use MockMvc or TestRestTemplate for this.
- **Consequences of Skipping:** Without tests, subtle bugs (especially in multi-tenant filters) may go unnoticed until demo. Not having tests also lowers code quality. Tests give confidence that features won't break on demo day.

Day 15 (Mar 18, 2026): Code Review and Refactoring

- **Objective:** Perform a thorough code review. Refactor any duplicate or messy code. Ensure coding standards and naming are consistent.
- **File Locations & Setup:** Review all Java packages and React src. Organize code: move classes if necessary, remove commented-out code, and improve logging statements.
- **Core Concepts:** Professional code should be clean and maintainable. This includes good naming, proper error handling (use @ControllerAdvice for global errors in Spring, for example), and adequate comments where needed. Also consider performance: add indices on tenant columns in the database if not already (done in Day 13 scripts).
- **Implementation:** Read through each file. Simplify logic where possible (e.g. use streams for list processing, or JPA queries instead of manual loops). Confirm application.properties does not contain hard-coded passwords (use environment variables or placeholders). Merge any overlapping services. Finally, fix any SonarLint/IDE warnings (null checks, etc.).
- **Consequences of Skipping:** Skipping review risks shipping unpolished or buggy code. It can leave inconsistencies that an interviewer might notice. Taking the time now will pay off in a stable, professional demo.

Day 16 (Mar 19, 2026): Backend Finalization & Deployment Prep

- **Objective:** Ensure the backend is production-ready. Prepare for packaging or deployment. Write deployment scripts if needed.
- **File Locations & Setup:** Create a Dockerfile for the Spring Boot app (in project root). Also, set up any continuous integration scripts (even if just simple shell scripts) to build and test the app.
- **Core Concepts:** In production, things like environment-based configs and build automation matter. Use Maven/Gradle to create an executable JAR (mvn package). The Dockerfile can use something like: FROM eclipse-temurin:17-jdk AS build to build, then a smaller runtime image. For example, copy the JAR into FROM eclipse-temurin:17-jre.
- **Implementation:** Write a basic Dockerfile (see Spring guide) that packages the JAR. Test building with docker build. Ensure the app runs inside the container (docker run -p 8080:8080). Document how to run tests and start the service in the README.
- **Consequences of Skipping:** If the backend isn't packaged, you can't reliably demonstrate deployment. Missing Dockerization means the environment must be configured manually, which isn't "production-ready." The interviewer expects at least a demo of containerization.

Day 17 (Mar 20, 2026): Frontend Setup and Data Integration

- **Objective:** Switch focus to the React frontend. Set up project structure and begin integrating with the backend APIs.
- **File Locations & Setup:** In the React app (/src), organize folders: e.g. components/, services/, contexts/. In services/, create an API client (e.g. using Axios) with a base URL to your Spring Boot API. In contexts/, put the TenantContext code (created on Day 7).
- **Core Concepts:** Ensure the front end can communicate with the backend. Handle CORS by configuring Spring Boot (already should allow origin http://localhost:3000). On the frontend, every API call must include the current tenant (e.g. attach X-Tenant-ID header from the React context to Axios requests). Remember that React is a Single-Page App, so routing will be via React Router (e.g. <BrowserRouter>).
- **Implementation:** Build the main page components: for example, a **WorkflowList** component that fetches GET /api/workflows on mount and displays them in a table. Use TenantContext to supply the tenant ID header: e.g. axios.get('/api/workflows', { headers: { 'X-Tenant-ID': currentTenant } }). Verify that the data shown is tenant-specific (test by switching tenant context manually and reloading).
- **Consequences of Skipping:** If the front end isn't connected to the backend, there is no visible UI for the functionality. Ensuring data flows to the UI is critical for the demo; skipping it means no output to show.

Day 18 (Mar 21, 2026): Frontend – Authentication & Protected Routes

- **Objective:** Implement or simulate a login flow on the frontend. Protect routes so only logged-in users see the app. Tie login to tenant context.
- **File Locations & Setup:** Create a Login component and possibly a PrivateRoute component (if using React Router) to guard pages. In contexts/, add an AuthContext to store user login state and tenant.
- **Core Concepts:** The front end should prompt for credentials or tenant name. Upon "login," store the token or just set a flag and tenant in state. Demonstrate that without logging in, the user is redirected to login. For simplicity, this can be a dummy login (e.g. entering tenantA as user sets the context). The TenantContext may be integrated here.
- **Implementation:** Develop a simple login form (username, password, tenant name). On submit, send a request to /api/login (if implemented) or simply set context locally. Then use React Router to redirect to the main dashboard. Ensure that all API calls now include the token/header from auth.
- **Consequences of Skipping:** A frontend without any authentication looks incomplete. Even if backend auth is not fully implemented, having a placeholder login form shows good design. If skipped, you must clarify in the demo that auth is stubbed.

Day 19 (Mar 22, 2026): Frontend – Workflow UI Implementation

- **Objective:** Build the main UI for viewing and manipulating workflows. This includes lists of workflows and tasks, and forms to create or update them.
- **File Locations & Setup:** In components/, add pages like WorkflowPage.js, TaskPage.js. Use state hooks and context. Ensure the app's entry (App.js) loads these routes.
- **Core Concepts:** Use React's component state or context to manage data. Design a clean UI (Material-UI or simple CSS) that shows data in lists or tables. Provide forms (using controlled components) to create a new workflow or task, tying into the backend endpoints (POST calls). Ensure the UI clearly indicates which tenant is active.

- **Implementation (cont.):** In WorkflowPage, fetch the list of workflows on load and display them. Include buttons or links to view a workflow's tasks. For example, clicking a workflow name navigates to /workflows/:id which shows its tasks. Implement the Task list page similarly. Each page should use the TenantContext to include the tenant ID header (e.g. configure axios defaults). Add functionality to delete or edit items (make DELETE/PUT requests).
- **Consequences of Skipping:** Without a functional workflow UI, the project has no visible feature. It's critical that the UI reflects the backend data and allows creating actions. Skipping this means no interaction for demo.

Day 20 (Mar 23, 2026): Frontend – Styling and Theming

- **Objective:** Polish the frontend appearance. Apply consistent styling, and support tenant-specific theming (colors, logos).
- **File Locations & Setup:** Use a CSS framework or library (e.g. CSS modules, Styled-Components) in your React project. If none chosen, manually add stylesheets under src/. Place any static assets (logo files, icons) under public/.
- **Core Concepts:** For multi-tenant apps, the UI often adapts to the tenant's branding (colors, logos). You can implement dynamic theming by, for example, setting a CSS variable or applying a different className based on the tenant context. Also ensure responsive layout and accessibility (e.g. ARIA labels).
- **Implementation:** Define a theme file (e.g. a JS or JSON object mapping tenant to color codes). In your TenantContext, after retrieving the tenant, apply a CSS class to the root (or set document body style) accordingly. Style the tables and buttons for readability. Verify that when tenant is switched (even manually), the theme changes (e.g. TenantA UI is blue, TenantB UI is green).

- **Consequences of Skipping:** A bland or inconsistent UI can detract from the demo. Poor styling might confuse users. While not core functionality, theming demonstrates attention to tenant-specific needs. Skipping styling makes the app look unfinished.

Day 21 (Mar 24, 2026): Frontend – Advanced Features (if Time)

- **Objective:** Add any remaining UI features, such as real-time updates or drag-and-drop workflow editing (if feasible). Otherwise, integrate simple enhancements.
- **File Locations & Setup:** Extend existing components or add new ones (e.g. a Dashboard component). If using WebSockets (optional), set up a Socket.io client in React to connect to the server for live updates.
- **Core Concepts:** Real-time feedback (via WebSockets) or workflow builder interfaces (using libraries like React Flow) can impress, but are optional. Focus on making sure CRUD is fully covered: after deleting a workflow, the list updates, etc.
- **Implementation:** For example, implement a refresh button or auto-refresh on a timer to show updated workflow status. Handle error messages gracefully (show alert on API error). Ensure forms validate input (no blank names, etc.). If any features mentioned in the design doc are incomplete, add them now.
- **Consequences of Skipping:** At this point, all core features should be done. Only skip if you've hit time. Missing nice-to-haves won't break the app, but ensure the basics are rock-solid first.

Day 22 (Mar 25, 2026): Hardening & Cross-Cutting Concerns

- **Objective:** Strengthen security, logging, and configuration. Prepare the application for deployment readiness.
- **File Locations & Setup:** In Spring Boot, review application.yml for prod vs dev profiles. Add logging statements (logger.info()) in important service methods. In the React app, set up production build scripts (npm run build).
- **Core Concepts:** Enable HTTPS if possible (or plan for it). Ensure CORS is tightly configured (only allow your frontend origin). Double-check that secrets (passwords, API keys) are not hard-coded. Consider adding Swagger/OpenAPI documentation (if comfortable) for the backend APIs as a nice bonus.
- **Implementation:** Configure @CrossOrigin on controllers or global CORS filter with your React origin. In production profile, set spring.datasource.url to the production DB (or a placeholder). Enable verbose logging for SQL only in dev. If time, add actuator or a /health endpoint.
- **Consequences of Skipping:** Hardening ensures the app doesn't have obvious flaws. Without it, reviewers might be concerned about missing security steps or inability to run in production. For a demo, at least explain what would be done (e.g. "we would set spring.profiles.active=prod and not commit secrets").

Day 23 (Mar 26, 2026): End-to-End Testing & Bug Fixes

- **Objective:** Perform end-to-end testing of the entire application. Find and fix any remaining bugs.
- **File Locations & Setup:** Optionally write end-to-end tests (e.g. with Cypress or Selenium). If not, manually go through every user flow: login as Tenant A, create a workflow, switch to Tenant B and ensure it's invisible, complete a workflow, etc.
- **Core Concepts:** Testing the integrated system is crucial. Cover common scenarios: no tenant header, invalid IDs, network errors. Watch for React state bugs (e.g. stale state after updates). Verify database state in Postgres after actions.
- **Implementation:** Run the React app (npm start) and the backend. In the browser, simulate two tenants in different windows. Fix any front-end bugs (e.g. component not re-rendering) and backend bugs (e.g. uncaught exceptions). If a critical issue is found, address it. Add more tests if needed to cover it.
- **Consequences of Skipping:** Unfixed bugs or missing flows will hurt the demo. This final testing ensures confidence. Failing to fix an obvious bug (like a 500 error on saving) could derail the entire presentation.

Day 24 (Mar 27, 2026): Dockerize Both Backend & Frontend

- **Objective:** Containerize the application for deployment. Create Docker images and a Docker Compose file to run everything together.
- **File Locations & Setup:** Ensure you have Dockerfile.backend (in root) and Dockerfile.frontend (inside React app). Write a docker-compose.yml to launch both containers (back and front) and a linked Postgres container.
- **Core Concepts:** Use best practices: multi-stage build for the Spring Boot app (e.g. using an official JDK image then a smaller JRE image). For React, build the static files (npm run build) and serve them with Nginx or a static server image. Docker Compose will orchestrate linking (set environment variable SPRING_DATASOURCE_URL to use the Postgres service).
- **Implementation:** In Dockerfile.backend, copy the compiled JAR and run java -jar. In Dockerfile.frontend, you may use node:alpine to build then serve. Define volumes and ports in docker-compose.yml. Test by running docker-compose up --build and accessing the app in browser. Ensure logs show both services starting.
- **Consequences of Skipping:** Demonstrating the app in Docker shows devops knowledge. If skipped, the app can still run locally, but containerization is a big plus for production-readiness. Not having a container could be noted as an "improvement point," so at least prepare to discuss it.

Day 25 (Mar 28, 2026): Final Testing & Debugging (Photo for Reminder)

- **Core Concepts:** Thoroughly test the containerized application. Verify that data persists correctly (Postgres volume) and that network links work. Check logs for errors. Ensure the correct ports are exposed (e.g. backend on 8080, frontend on 3000 or 80 if served by Nginx).
- **Implementation:** Run the dockerized app as a whole. Try shutting down one container and restarting to ensure the system comes back cleanly. If any 404s or errors appear in the browser console, fix routing or assets issues. Retest all features one more time in the container environment.
- **Consequences of Skipping:** Any remaining docker or deployment bugs will break the final demo. This day's focus ensures the container build is solid. Skipping this could lead to deployment failures or "it works on my machine" issues during presentation.

Day 26 (Mar 29, 2026): Final Review & Demo Preparation

- **Objective:** Prepare presentation materials and practice explaining the project. Ensure readiness for questions.
- **Core Concepts:** Summarize what was built: multi-tenant backend with RLS for security, React frontend with context, workflow automation features, Docker deployment. Understand why each technology was chosen.
- **Implementation:** Create a short slide deck or bullet list of talking points: project goals, architecture diagram, key challenges (and solutions), demo steps. Rehearse the demo path: logging in, creating workflows, switching tenants to show isolation, etc. Make sure your workstation is set up for the demo (all services running or containers up).
- **Consequences of Skipping:** Without rehearsal, even a working project can appear unreliable. Being unprepared to discuss your design or code can undermine confidence. This final step ensures you can clearly explain and demonstrate every aspect of the project as expected in an interview.